

Real Time Crack Detection using Simultaneous Location and Mapping (SLAM) and Deep Learning for Structural Health Monitoring (SHM)

Adam Camerer and Reece Watson

CS 6604

Prof. Sanjay Madria

10 May 2025

Abstract-This paper presents a unified, real-time crack detection and 3D localization pipeline for concrete infrastructure inspection that is optimized for edge devices. We apply multi-sensor SLAM, image super-resolution, lightweight segmentation, and crack refinement to enable accurate, field deployable structural health monitoring. An adaptive normal-tangent PCA algorithm estimates subpixel crack widths and a backprojection module reconstructs crack positions in 3D space. We evaluate performance across synthetic and benchmark datasets, with experimental results demonstrating the system's feasibility and efficiency for real-time applications such as handheld scanning and autonomous drone inspection. Our modular implementation provides a foundation for future adaptations and conversion to ROS2.

1 | Introduction

Infrastructure deterioration poses a significant challenge to public safety, economic stability, and sustainability. Bridges, tunnels, dams, and industrial facilities experience gradual degradation due to environmental exposure, structural stress, and natural disasters. Concrete based construction is often the backbone of modern infrastructure, with its material availability, cost effectiveness and strong durability allowing it to be the backbone of modern infrastructure [1]. Crack detection, originating on and observable on the surface of concrete, are easy and reliable indicators of a structure's stress, safety and wear, and are essential metrics in assessing structural health during inspections [1]. The width and number of cracks are critical markers for assessing structural integrity, and the spatial distribution and orientation of cracks can provide insights into whether the cracks are structural or non structural, as well as their causes [3]. Complex textures, varying lighting conditions, and stains or spots can make it challenging to detect cracks, and the appearance of small and narrow cracks can be indistinguishable from background noise. Finally, the quality of images can vary due to factors such as resolution,

focus and capture angle, necessitating the need for robust and generalizable methods for crack inspection [2].

For automated, comprehensive and practically meaningful concrete crack inspection to be achieved, three requirements must be fulfilled: efficient capture of 3D spatial characteristics, high-precision detection at the submillimeter level, and a generalizability across various scenarios of crack detection [1]. Furthermore, crack analysis that can be processed on site and preferably in real time can serve as an early alert to inspectors, allowing for refined analysis on problem areas while on site that can result in a greater degree of preventative measures.

Traditional inspection methods rely on manual assessments, stationary sensors, or periodic data collection, which often fail to detect early warning signs of structural failure [1]. Manual inspections are not only time-consuming and expensive but also introduce the risk of human error, making them inefficient for large-scale infrastructure monitoring [1]. The need for a more accurate and automated solution has led to the development of advanced remote sensor based technologies. 3D reconstruction of a structure is often used to search for deformities in a bridge and searching for creating orthoimages of a crack region, which can then be used to measure the width of said crack, with image and LiDAR based systems playing crucial roles in 3D reconstruction [2]. Structure from Motion (SfM) is a common camera-based method for reconstructing a 3D model, and requires that the captured images have a large overlapping area and precise trajectory planning to control the working image, from which photogrammetry techniques generate high precision point clouds for 3D reconstruction [4]. However, SfM requires meticulous trajectory planning and extended computing time when dealing with large amounts of images, as well as only being able to create point clouds at the centimeter level when cracks are often at the millimeter level [4]. As a result, it is not effective to utilize SfM since crack information cannot be directly obtained from the 3D model.

Simultaneous Location and Mapping (SLAM), a technique for trajectory estimation of objects and 3D reconstruction of scenes has emerged as a superior alternative [2]. By combining IMU, LiDAR and camera data, camera data can be used for defect inspection, while LiDAR and IMU data for accurate localization using the current state of the art tightly coupled IMU-Vision-LIDAR FAST-LIVO2 multi sensor fusion algorithm can allow for high-precision surface textures and sufficient information for crack localization. While previous work has demonstrated adequate performance in comparison to SLAM, it is still relatively computationally costly, especially the work of Deng et al. which utilizes DeepLabV3+ and SAM for few shot crack segmentation [1]. Our goal is to develop an algorithm that demonstrates comparable or superior performance to prior work while being computationally efficient enough to be directly usable in

the field for a handheld or drone based approach. Specifically, the novelty and contribution of this paper is:

1. A handheld 3D crack inspection system that utilizes multi sensor calibration and integrated data processing on a NVIDIA Jetson Xavier NX (rudimentary prototype)
2. Utilizing state of the art super-resolution with WaveMIXSR-V2 for improved visibility of subpixel cracks
3. An efficient few-shot concrete segmentation method utilizing specialized crack transformer SCSegamba for automated, generalized crack detection and an adapted Efficient-ViT-SAM-L0 as an efficient version of the refinement tool used in Deng et al's paper [5] for crack detection.
4. Improved crack width estimation with adaptive tangent-normal pca analysis
5. 3D crack localization and marking to show extent as well as depth possibly.
6. Integration of said systems into an ROS2 framework for modularity and integration for future autonomous navigation.

2 | Literature Review

2.1 | Multi Sensor SLAM

SLAM algorithms have made significant advancements in the last 36 years since their initial inception with the LOAM algorithm and Kalman filters. SLAM has been extremely useful for its ability to estimate poses for trajectory estimation and real-time 3D reconstruction and localization in unknown environments [5]. SLAM primarily relies on a combination of LiDAR, camera and IMU data for localization and mapping, with IMU allowing for accurate estimations of short term movement, LiDAR being able to calculate precise depth measurements without the need for an external light source, and cameras being using RGB information for rich semantic insights, with binocular cameras being able to achieve a slightly less accurate level of depth perception at a significantly lower cost [5]. Each of these individual components has their own weaknesses in 3D map reconstruction, with IMU suffering from bias and long term drift, vision from the lack of direct depth estimation necessitating triangulation or depth filtering, sensitivity to low light levels and glare, as well as requiring calibration, while LiDAR lacks color information and deteriorates in environments with insufficient geometric constraints such as narrow tunnels and extended walls [5]. To be able to accurately operate robots in the real environment, algorithms taking advantage of multiple sensors is needed, where the strength of each sensor is utilized to overcome the weakness of each sensor to result in accurate, dense and colored 3D point clouds maps. SLAM algorithms can be classified by the sensors used (Lidar, Visual,

Inertial, and / or Odometry), and the degree of ‘tightness’ for each of the components. A loosely coupled system processes measurement data separately and then fuses them in a filter marginalizing the current frame to acquire the latest state estimate results [5]. A tightly coupled system on the other hand jointly optimizes the measurement data of all sensors and combines the observation characteristics and physical models of each sensor to more robustly estimate the pose [5]. While loosely coupled systems require less calculations, have a simpler system and are easier to implement, a tightly coupled system is more computationally intensive but has a more accurate state estimation in complex and changeable environments [5].

In recent years, tightly coupled LIVO systems have been leading the field with superior performance and accuracy while being able to operate in real-time with limited computation in the field. One of the current state of the art systems using such a system is FAST-LIVO2, which uses this in conjunction with an error-state iterated Kalman filter (ESIKF) to more efficiently run mapping and planning with superior results compared to other methods such as R3LIVE and LVI-SAM, with R3LIVE being used in Feng et al.’s 2023 crack detection implementation [2,5].

2.2 | Crack Segmentation with Deep Learning

In the last few years, deep learning models, particularly convolutional neural networks (CNNs) and transformers, have revolutionized crack segmentation. CNN-based approaches like DeepCrack and U-Net variants have shown superior accuracy over thresholding by learning crack features from data [12]. Yet, CNNs can struggle to capture the global context needed for very fine or diffuse cracks. Transformer-based architectures CrackFormer have recently emerged as state-of-the-art for crack segmentation, using self-attention to better delineate cracks from. CrackFormer introduced a transformer network for fine-grained crack detection, using self-attention in a SegNet-like encoder-decoder architecture to better delineate cracks from background [11]. An improved version, CrackFormer-II, was later developed to generalize across more diverse crack datasets, its encoder fusing multi-scale features with self-attention to outperform CNN models and achieve top performance on datasets like Crack500 at its publication [11]. Further studies utilized another transformer, Segment Anything Model (SAM), with a specially modified adapter SAM-Adapter and a Mixture of Gaussian-based (MoG) class distribution modeling to dramatically improve the performance of SAM at Crack Segmentation performance, surpassing even CrackFormer-II [11]. SAM, a transformer-based foundation model for segmentation, has strong generalizable zero-shot capability. SAM-Adapter uses the learning module Adapter to incorporate domain specific information into the model, where the learnable

parameter Adapter conveys information through the formula $P_i = MLP_{up}(GELU(MLP_{tune}^i(F_i)))$, where MLP_{tune}^i represents the i^{th} linear layer that generates task-specific hints for each adapter, MLP_{up} represents the upper projection layer used to adjust the feature dimension of the converter. F_i can be various forms, such as, the texture or frequency or combination. P_i is a hint generated by the Adapter to guide the model to task-specific adjustments in each layer of converters [12]. Statistically, MoG can depict the semantic distribution of various crack types more accurately [12]. These transformer models can detect very thin cracks and complex shapes under difficult imaging conditions (poor lighting, shadows) because the self-attention layers aggregate information from a broad context, reducing sensitivity to local noise [12]. Despite these advances, one challenge is that training such models typically requires a lot of labeled crack data, and may not generalize well to other models, also known as domain shift [12].

Deng et al (2025) introduced a few-shot segmentation pipeline utilizing foundation models to improve generalization, with the base model DeepLabv3+ being trained on a small crack dataset, and its output being refined by SAM, significantly boosting segmentation recall and adaptability to new scenarios [1] This two-stage approach addressed the issue of incomplete masks from the base model by harnessing SAM's broad visual knowledge, but is computationally heavy for real-time systems, requiring a quality assessment filter to remove any spurious segments or adapting / fine-tuning SAM for cracks (i.e. SAM-Adapter), injecting crack-specific knowledge while retaining generalization. The literature suggests that combining large pre-trained models with crack-specific training can yield highly robust segmentation that works in diverse environments [13], and we hope to make a higher performance and significantly more efficient system based on Deng et al's (2025) process that is specifically designed for crack detection, achieving generalization without needing on-the-fly prompt refinement.

2.3 | 3D Crack Mapping and Measurement

Beyond 2D detection, there is a growing focus on obtaining the *3D characteristics of cracks* – such as their true width, depth, orientation, and spatial location on the structure. Early methods estimated crack width in 2D images and then converted to metric units using a scale [1]. This assumes the crack lies roughly on a plane perpendicular to the camera, which is often not true on curved surfaces. The 2023 multi-sensor approach by Feng *et al.* produced a colored 3D point cloud of the structure via SLAM, but crack measurements were still done per image and then associated with the 3D model manually, measuring only crack length using bundle adjustment, which is prone to drift [2]. In contrast, Deng *et al.* (2025) fully unified the process:

their multimodal SLAM (using both camera and LiDAR) created a dense 3D point cloud of the structure, and they projected the crack masks onto this 3D model to measure crack properties directly in the point cloud space [1]. By analyzing cracks in 3D, they could handle curved or angled structural surfaces where 2D image measurements would be skewed or inaccurate [1]. Their system performs multi-frame fusion, meaning if a crack is seen in multiple images, the information is merged to yield a more complete crack representation on the 3D model [1]. This improves detection of cracks that might be partially visible in any single image and reduces false positives by cross-verifying across views. The result is an automated crack mapping with each crack's location, length, and width quantified in real-world units on the as-built structure [1]. Such capabilities are critical for inspecting complex geometries (arches, pillars, etc.), where understanding the 3D orientation and extent of a crack guides structural assessments.

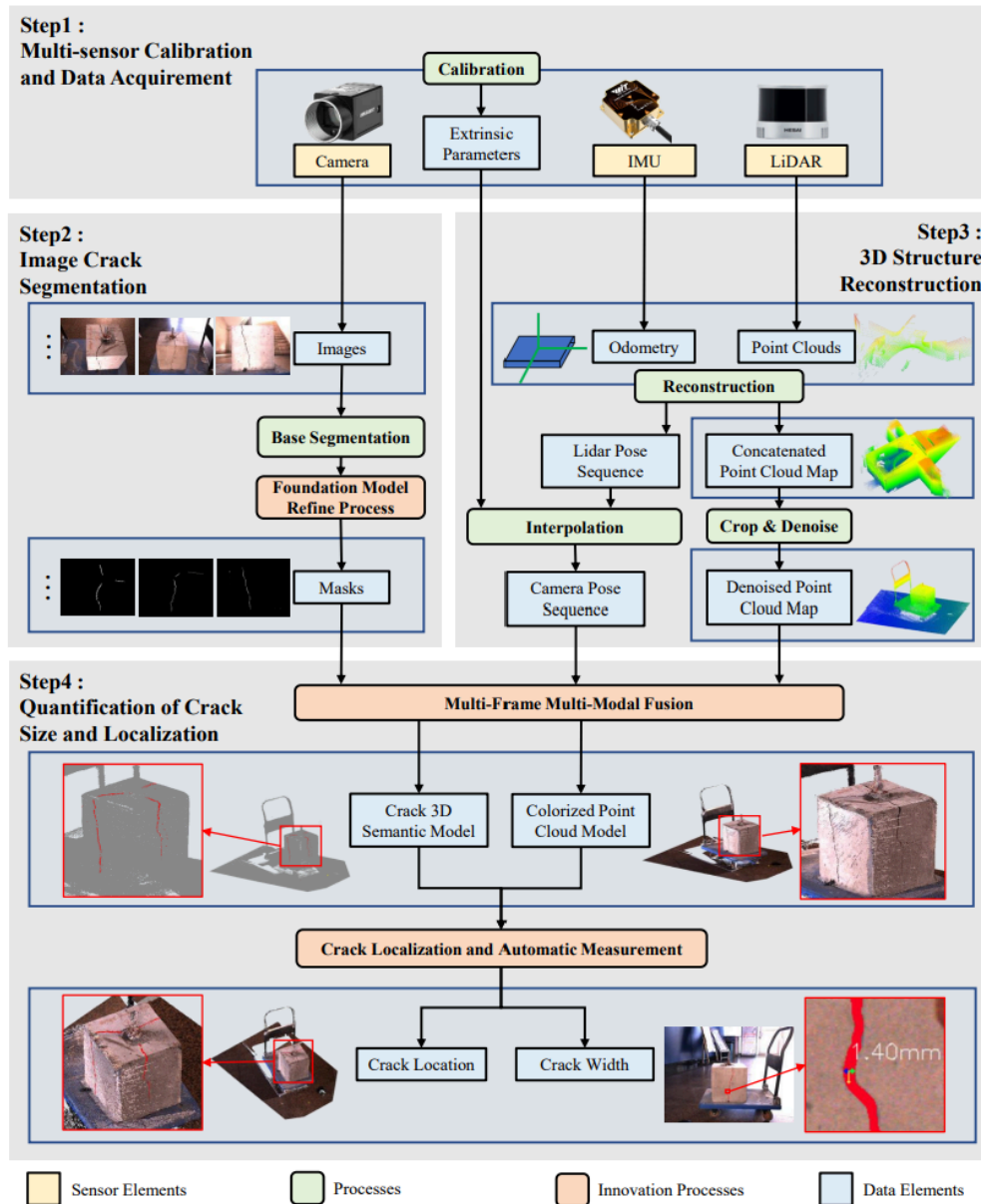


Figure 1: The framework used in Deng et al (2025). The sensors are calibrated, then images are passed through semantic segmentation while LiDAR generates point clouds. Using a fusion module and post-processing, automatic crack detection and reconstruction are achieved

2.4 | Gaps in current literature

While prior work has demonstrated the potential for SLAM and deep learning based segmentation for crack detection, there is significant room for improvement. Feng et al (2023). showed real-time mapping and improved crack detection with super-resolution [2], but lacked a learning-based segmentation. Deng *et al.* introduced advanced segmentation with a foundation model refinement and achieved precise 3D crack measurements [1], but their approach is too complex and slow for real-time use on edge devices. There remains a need for a unified system to build on these advances and optimize for efficiency.

3 | Architecture

3.1 | Overview

Building from Feng et al's 2023 paper and Deng et al's 2025 paper, our pipeline utilizes a similar framework compared to the previous approaches. The initial section of our pipeline utilize the same framework but use refined and optimized components, replacing older SLAM models R3LIVE and FAST-LIO2 used by each paper respectively with the improved FAST-LIVO2, Real-ESRGAN AI upscaling in Feng's paper with WaveMix-SR-V2, Deng's effective but slow DeepLabV3+B and the base SAM two-stage crack-segmentation with SCSegamba for initial segmentation and EfficientViT-SAM-L0 for refinement. The latter section of our pipeline utilizes custom made algorithms improving upon the previous works, utilizing adaptive normal tangent PCA instead of uniform PCA to estimate the width of the crack, followed by an advanced 3D backprojection algorithm to integrate the crack width measurements with the SLAM-generated point cloud for precise spatial localization. This approach not only enhances segmentation accuracy through dynamic refinement but also improves computational efficiency, making it suitable for real-time deployment on edge devices such as the Jetson Xavier NX.

3.2 | Multi-Sensor SLAM with FAST-LIVO2

The system primarily consists of an ESIKF, a LiDAR Measurement Model, and a Visual Measurement Model [5]. FAST-LIVO2's main innovations are:

- 1) An efficient ESIKF framework to address the mismatch between LiDAR and camera.
- 2) Using plane priors from LiDAR to improve accuracy.
- 3) Selecting high quality, inlier reference patches with large parallax and sufficient reference details.

- 4) Conducting online exposure time estimation for environment illumination variation.
- 5) On demand voxel raycasting in the absence of LiDAR measurements.

Compared to R3LIVE, which operates at the individual pixel level and uses a dense direct method for the VIO, FAST-LIVO2 uses a sparse direct method operating at the patch level that offers superior performance, among other enhancements [5].

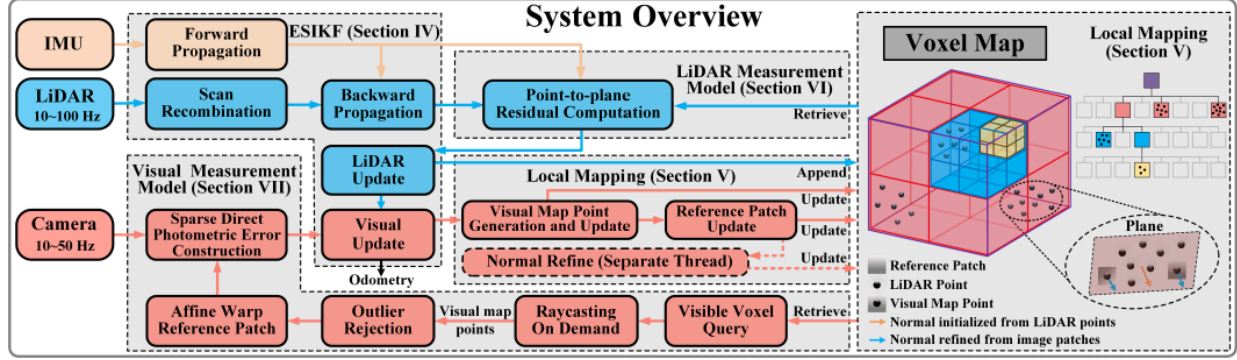


Figure 2: System overview of FAST-LIVO2

The state transition model is modeled as $x_{i+1} = x_i + \Delta t f(x_i, u_i, w_i)$, where Δt is the IMU sample period, the state is x , the input is u , the process noise is w , and the function is f [5].

The uncertainty of a LiDAR point in the local frame is decomposed into the ranging uncertainty δd caused by the laser time of flight (TOF) and the bearing direction $\delta \omega$ originated from encoders. δd from the laser angle divergence can be modeled as

$$\delta d = L_2 - L_1 = d \left(\frac{\cos \rho}{\cos(\theta + \rho)} - \frac{\cos \rho}{\cos(\theta - \rho)} \right),$$

where ρ is the angle between the bearing direction and normal vector, and θ is the laser beam divergence angle [5].

For updating the reference patch, each patch f is scored on photometric similarity and viewing angle via

$$NCC(f, g) = \frac{\sum_{x,y} [f(x,y) - \bar{f}][g(x,y) - \bar{g}]}{\sqrt{\sum_{x,y} [f(x,y) - \bar{f}]^2 \sum_{x,y} [g(x,y) - \bar{g}]^2}},$$

where $NCC(f, g)$ is the normalized cross-correlation used to measure the similarity between patch f and g at the 0-th pyramid level [5].

3.3 | Real-Time Crack Image Enhancement with DSCLoRA (Distillation-Supervised Convolutional Low-Rank Adaptation)

For high-fidelity SISR, DSCLoRA captures both fine-grained local details and global context by replacing SPAN's parameter-free attention blocks with SConvLB modules, which integrate ConvLoRA layers into both standard convolution operations and the pixel shuffle upsampling block [5]. By freezing the original pre-trained weights and updating only the low-rank decomposition matrices X and Y , DSCLoRA enhances representational capacity without increasing inference complexity or parameter count [5]. DSCLoRA employs spatial affinity-based distillation across multiple feature layers alongside pixel-level and reconstruction losses, effectively transferring second-order statistical knowledge from a robust SPAN teacher model [5]. The multi-stage design leverages progressive upsampling via ConvLoRA-enhanced pixel shuffle blocks, with Exponential Moving Average stabilization and data augmentation for stable convergence on varying patch sizes [5]. DSCLoRA achieves state-of-the-art PSNR and SSIM on benchmarks like BSD100 and Manga109 while maintaining under 0.14 M parameters and sub-10 ms inference time for $4\times$ SR on 256×256 inputs, demonstrating superior efficiency compared to transformer-based and GAN-based methods [5]. As the winner of the NTIRE 2025 EfficientSR challenge, we selected this model for being the current SOTA in efficient, real time super resolution, being much more effective than WaveMixSR-v2, initially selected due to its high performance on the BSD100 dataset while having near real-time performance, whose pretrained model demonstrated poor performance compared even with methods like Lanczos and Bicubic. DSCLoRA's parameter-efficient adaptation yields significant improvements in super resolution compared to Real-ESRGAN while being over 17x more efficient, based on testing done on the DIV2K dataset.

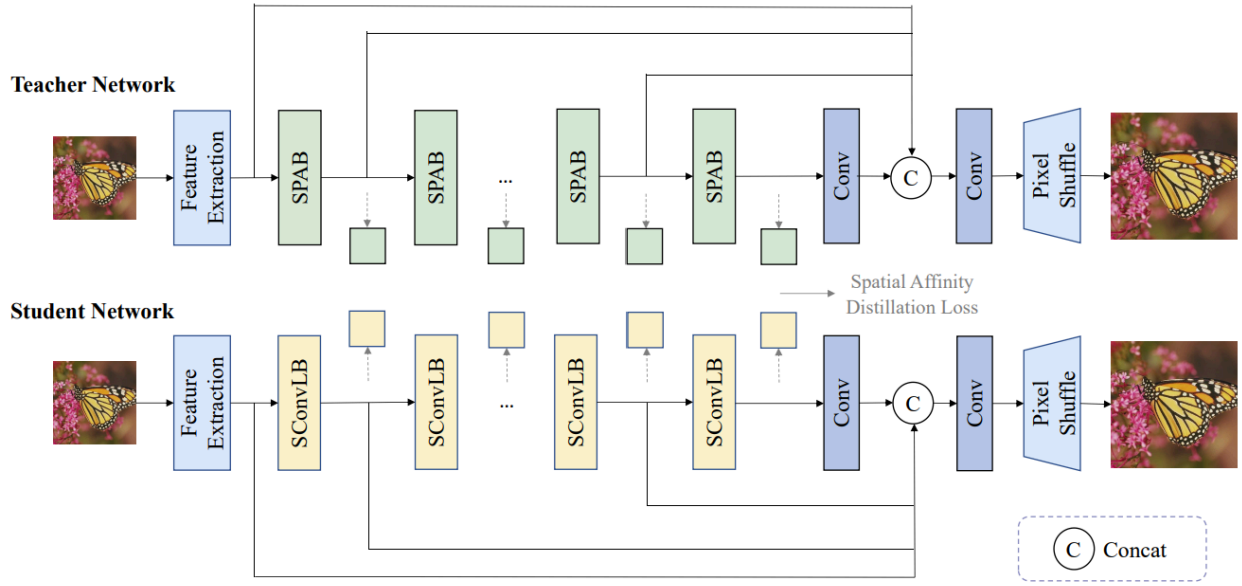


Figure 3: Architecture of DSCLoRA. The top row depicts the **teacher network**, which consists of a feature-extraction backbone followed by a series of SPAB modules; intermediate feature maps from each SPAB block are used to compute spatial affinity distillation losses (gray arrows). The bottom row shows the **student network**, where each SPAB is replaced by the proposed SConvLB module—integrating ConvLoRA layers into both its convolutional operations and the pixel-shuffle upsampling block—and multi-stage features are concatenated (black arrows) into a final convolution before pixel-shuffle to produce the 4× super-resolved output.

3.4 | Crack image detection with SCSegamba (Structure Aware Vision Mamba)

SCSegamba is a lightweight Structure-Aware Vision Mamba network designed to generate high-quality pixel-level crack segmentation maps with minimal computational resources [6]. It introduces a Structure-Aware Visual State Space (SAVSS) module that integrates a Gated Bottleneck Convolution (GBC) and a Structure-Aware Scanning Strategy (SASS) to enhance the perception of crack morphology and texture [6]. Within each SAVSS block, selective state updates follow $z_k = Pz_{k-1} + Qw_k$ and $u_k = Rz_k + Sw_k$, where P and Q capture multi-directional spatial dependencies from parallel and diagonal snake scans, and R and S project hidden states and inputs [6]. Pixel Attention-oriented Fusion (PAF) merges original and scanned feature sequences via gated concatenation to preserve fine-grained crack details [6]. After SAVSS stages, a Multi-Scale Feature Segmentation Head (MFS) applies lightweight MLPs and dynamic upsampling to each multi-resolution feature map, concatenates them, and refines the result with a final convolutional layer to produce the segmentation output [6]. A hybrid objective combining Dice and BCE losses balances boundary precision, context

consistency, and class imbalance, ensuring robust performance across diverse crack scenarios [6].

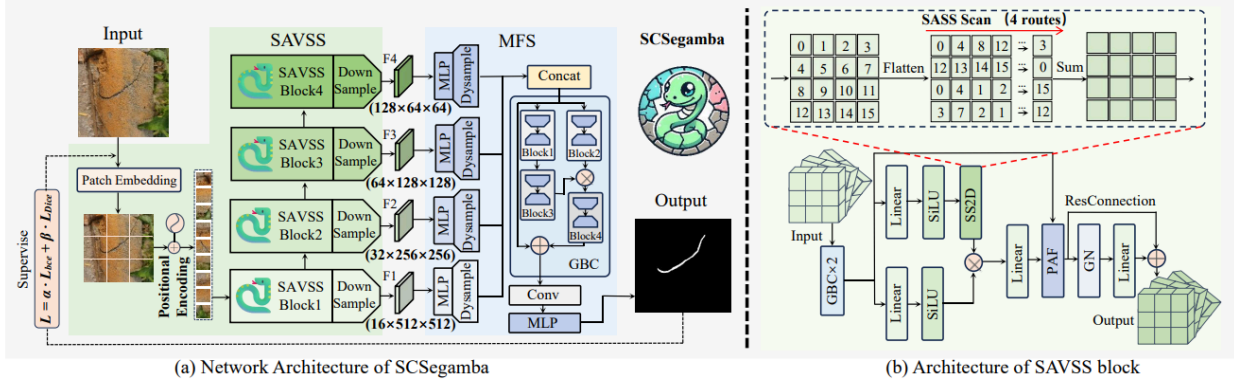


Figure 4: Overview of SCSegamba architecture. (a) illustrates the overall architecture of SCSegamba and the processing flow for crack images. (b) displays the structure of the SAVSS block. The input crack image undergoes comprehensive morphological and texture feature extraction through SAVSS, while MFS produces a high-quality pixel-level segmentation map.

3.5 | Refinement with EfficientViT-SAM-XL0

After the release of Meta's Segment Anything Model (SAM) in 2023, there has been a lot of research to develop more efficient models, especially for real time usage on edge devices. In a recent 2024 paper, eight different versions of SAM from the original to newer renditions like MobileSAM, EfficientSAM, SlimSAM, TinySAM, NanoSAM and Efficient-ViT-SAM were examined, comparing throughput and mIoU for a variety of datasets [8]. While NanoSAM had the highest throughput, Efficient-ViT-SAM-L0 had considerably higher performance while still retaining a very high throughput, thus resulting in us choosing Efficient-ViT-SAM for our generalized segmentation refinement model [8].

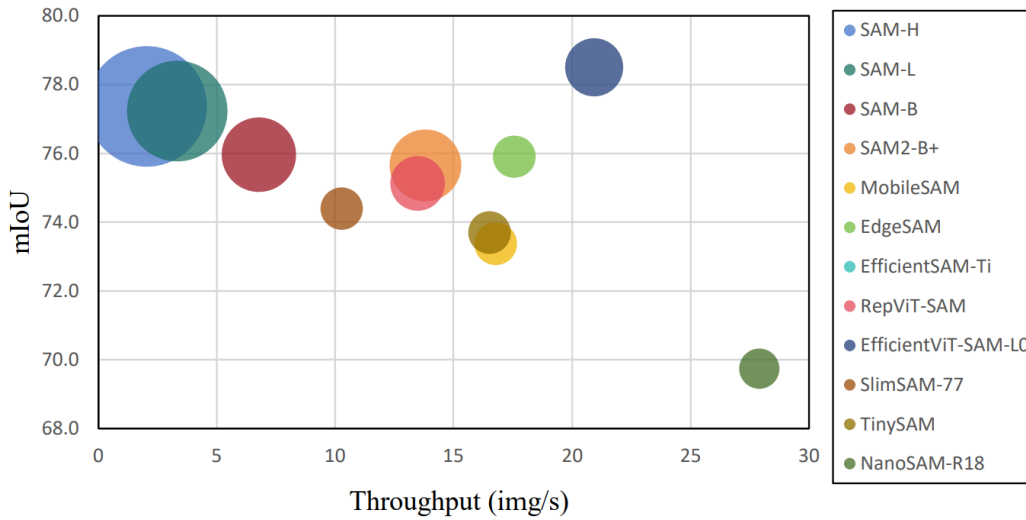


Figure 5: model comparison study results done by Sun et al. in 2024.

After SCSegamba produces an initial segmentation mask, the output is passed as a prompt to EfficientViT-SAM-L0. This module leverages an efficient vision transformer (ViT) backbone, substituting the inefficient softmax attention with lightweight ReLU linear attention to reduce the complexity from quadratic to linear time while preserving functionality, and enhancing the ReLU linear attention with convolution to mitigate its limitation in local feature extraction. [9]. EfficientViT-SAM-L0 is the most compact model of the EfficientViT-SAM family, boasting only 34.8 million parameters compared to SAM's 641.09 while having much higher performance and comparable efficiency to models like EdgeSAM [9]. It recalibrates the initial mask, refining edges and reducing false positives by capturing both global context and fine local details. Compared to SAM, EfficientViT-SAM delivers 48.9× measured TensorRT speedup without sacrificing performance, making it suitable for gpu accelerated edge devices like the Jetson Xavier NX [9]. Integrating this module results in a refined crack mask that significantly improves the accuracy of downstream processing, especially in scenarios with low-contrast or occluded cracks. The L0 model demonstrated very poor no-shot segmentation of the crack mask, and while the XL0 model, which is about 3 times heavier, offered significantly better performance, and ran on a wider 1024x1024 input window size instead of SCSegamba's limited 512x512 input size, allowing for wider tiling. It still needed fine tuning to compensate for inadequate performance compared to SCSegamba.

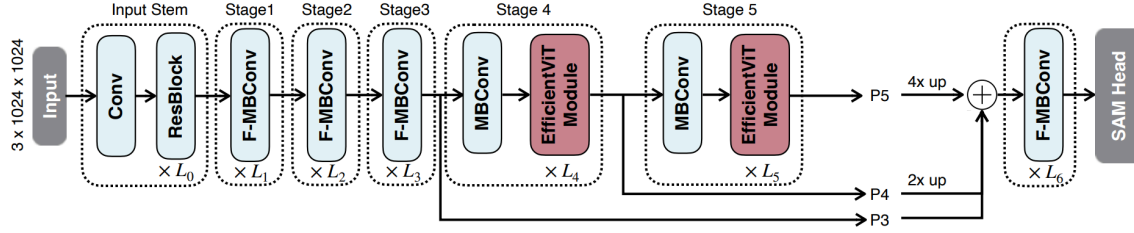


Figure 6: macro architecture of Efficient-ViT-SAM-XL. ResBlock is building block of ResNet34. F-MBConv is fused MBConv block of EfficientNetV2.

3.6 | Adaptive normal-tangent PCA-based crack width estimation

We improved upon Deng. et al (2023), which used a uniform fixed-size window normal-tangent PCA based method to replace it with one that adaptively adjusts the window-size and sampling resolution based on local image quality and segmentation confidence. We first skeletonize the points using opencv's skeletonize method. At each skeleton point, PCA is performed on its adaptive neighborhood to estimate the local tangent, and the corresponding normal is computed. Rather than projecting a fixed length along the normal, the algorithm uses gradient information from the segmentation logits to adjust the step size dynamically, achieving sub-pixel edge localization. This dynamic behavior is mathematically analogous to the state update equation $h_{k+1} = (I + \Delta A)h_k + \Delta Bx_k$ and the adaptive parameterization $B = S_B(x)$, which together ensure that the edge sampling is responsive to the local crack features. This approach draws inspiration from Feng et al. (2023) and Deng et al. (2025), who both highlighted the need for enhanced segmentation precision, as well as from the insights provided in the Mamba meets Crack Segmentation study, which demonstrated that integrating dynamic attention with traditional state-space models can greatly improve the accuracy and efficiency of crack width estimation [1,2,6].

3.7 | 3D Backprojection and Crack Localization with Uncertainty Estimation

To accurately map detected cracks into the real-world 3D coordinate frame, we employ a backprojection pipeline that combines 2D crack segmentation results with depth data and SLAM-based pose estimation. This allows for spatially coherent mapping of crack pixels, including width and orientation, onto the reconstructed 3D surface, enabling both metric quantification and visual localization, and is largely similar to Deng et al's approach [1].

The process consists of four key steps: (1) pose retrieval from FAST-LIVO2, (2) segmentation and pixel selection via SCSegamba, (3) depth-assisted 3D backprojection, and (4) uncertainty quantification via pose perturbation.

1. Camera Pose Estimation:

The FAST-LIVO2 framework provides a real-time camera-to-world transformation matrix $T_{cw} \in SE(3)$, representing the rigid-body pose of the camera at the time of image capture.

The transformation consists of a rotation matrix $R_{cw} \in SE(3)$ and a translation vector $t \in \mathbb{R}^3$, such that:

$$\mathbf{T}_{cw} = \begin{bmatrix} \mathbf{R}_{cw} & \mathbf{t}_{cw} \\ \mathbf{0} & 1 \end{bmatrix}$$

This matrix is obtained from FAST-LIVO2's visual-inertial-LiDAR backend, which uses an error-state iterated Kalman filter (ESIKF) to tightly couple sensor measurements and maintain drift-free localization in real time [8].

2. Crack Segmentation:

SCSegamba is applied to the high-resolution image (after super-resolution enhancement) to produce a segmentation logit map $M \in \mathbb{R}^{H \times W}$, followed by refinement using EfficientViT-SAM. Crack pixels $p = (u, v)$ satisfying $M[u, v] = 1$ are selected for 3D reconstruction [7].

3. Backprojection to 3D:

For each valid crack pixel p , the corresponding 3D point in the world frame is computed through a calibrated projection process: a) Compute normalized image coordinates using the intrinsic matrix $K \in \mathbb{R}^{3 \times 3}$:

$$\hat{\mathbf{x}}_{cam} = \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

b) Estimate the depth value d either from synchronized LiDAR or by intersecting with a fitted surface plane. c) Reconstruct the 3D point in the camera frame:

$$X_{cam} = d \cdot \hat{\mathbf{x}}_{cam}$$

d) Convert to world coordinates using the inverse transformation:

$$X_{world} = R_{wc} \cdot X_{cam} + t_{wc}$$

where $R_{wc} = R_{cw}^T$ and $t_{wc} = -R_{cw}^T \cdot t_{cw}$.

This approach is grounded in classical pinhole camera geometry [14] and adapted to real-time SLAM pipelines as demonstrated in previous works [1], [2].

4. Uncertainty Quantification:

Given that SLAM pose estimates are subject to drift and sensor-induced noise, we model the transformation T_{cw} as a random variable perturbed by a zero-mean Gaussian noise:

$$\mathbf{T}_{cw} \sim \mathcal{N}(\mathbf{0}, \Sigma_T)$$

where Σ_t is derived from FAST-LIVO2's internal covariance estimator [8]. To quantify how pose uncertainty affects 3D crack localization, we generate N perturbed samples $T_{cw}^{(j)}$ and repeat the backprojection for each:

$$\mathbf{X}_{\text{world}}^{(j)} = f(\mathbf{T}_{cw}^{(j)}, \mathbf{K}, \mathbf{p}, d)$$

The uncertainty of the crack point X_i is then captured via sample covariance:

$$\mathbf{U}_i = \text{Cov} \left(\left\{ \mathbf{X}_{\text{world}}^{(j)} \right\}_{j=1}^N \right)$$

This provides a per-point estimate of spatial variance and can be visualized as ellipsoidal confidence regions or used to reject high-uncertainty crack points.

This backprojection and uncertainty framework allows us to map crack edges and widths from 2D to 3D, enabling metric-space evaluation and integration into SLAM-based maps, much like the pipeline described by Deng et al. [1], but optimized here for real-time performance.

4 | Implementation

The proposed system integrates SLAM, super-resolution, segmentation, crack width estimation, and 3D backprojection into a unified, modular pipeline designed for real-time structural health monitoring on embedded platforms. The design prioritizes computational efficiency and generalizability while maintaining accuracy.

4.1 | System Pipeline Overview

The core implementation is structured as follows:

1. Sensor Input and Calibration

Multi-sensor input is collected from a camera, LiDAR, and IMU system. Calibration prior

to runtime ensures alignment between coordinate frames, enabling consistent data fusion. All sensors are time-synchronized for real-time processing.

2. SLAM and Pose Estimation

We adopt FAST-LIVO2, a tightly coupled, direct multi-sensor SLAM framework that fuses LiDAR, IMU, and visual data using an error-state iterated Kalman filter. This module provides high-precision pose estimation and sparse 3D mapping, forming the geometric backbone of the system.

3. Image Super-Resolution

Input RGB images are enhanced using DSCLoRA, doing efficient 4x image super-resolution. The improved image clarity enables the detection of thin, subpixel cracks, especially in challenging lighting or low-resolution scenarios.

4. Crack Segmentation and Refinement

The enhanced images are passed through SCSegamba, a transformer-inspired state space model optimized for efficient and robust crack segmentation. The initial binary mask is then refined using EfficientViT-SAM-L0, a lightweight segmentation module based on efficient ViT backbones with linear attention.

5. Crack Width Estimation via Adaptive PCA

Skeletonized crack masks are analyzed using an adaptive normal–tangent PCA algorithm, which dynamically adjusts window sizes and projection steps based on local segmentation confidence and image gradients. This method provides sub-pixel crack width measurements, improving over traditional fixed-window PCA approaches.

6. 3D Backprojection and Localization

Crack pixels are backprojected into world coordinates using depth data (from LiDAR or estimated planes), camera intrinsics, and pose estimates from FAST-LIVO2. Spatial transformation is applied to obtain real-world 3D positions of crack points. Uncertainty is quantified via repeated backprojection with sampled pose perturbations, generating confidence estimates for each 3D crack point.

All modules are designed with a modular architecture to support integration into a ROS2-based system, facilitating future extensions such as autonomous navigation, multi-agent collaboration, or hardware-specific optimizations for embedded deployment.

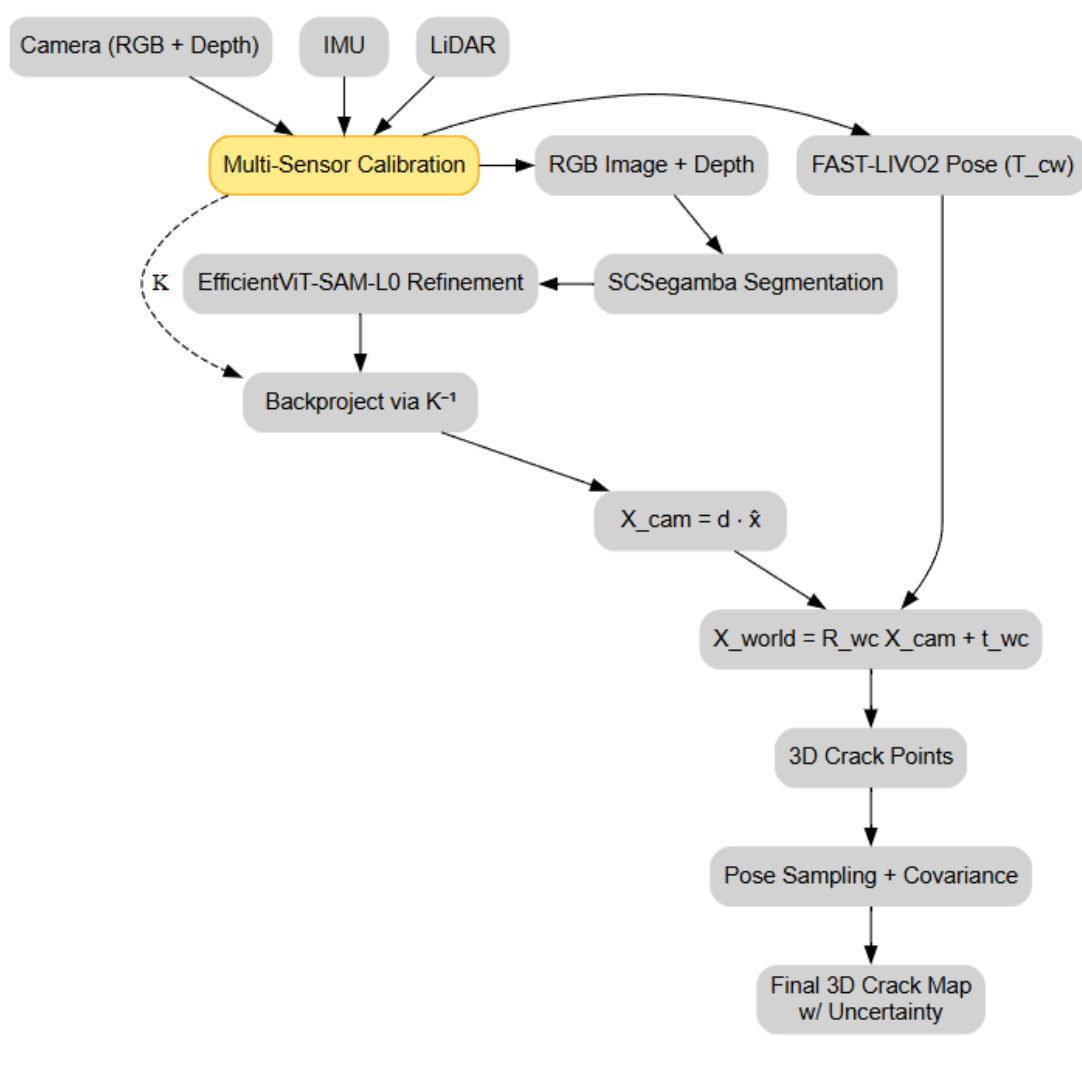


Figure 7: 3D crack localization and uncertainty-aware backprojection pipeline. Crack pixels are backprojected into world space using SLAM pose and intrinsics. Uncertainty is estimated via pose perturbation and covariance analysis.

4.2 | Pipeline Overview Pseudocode

Below is a pseudocode sketch of a high level overview of our crack width estimation pipeline from start to finish.

```

# Initialize and configure all necessary modules and models
calibrated = false
calibrated_parameters = []

```

```

#calibrate upon initialization, or import from pre-existing calibration
calibrated_parameters_path = /path/to/file
use_preexisting_calibration_settings = true
if not calibrated_parameters_path.isfile():
    calibrated_parameters = initialize_calibration()
    if use_preexisting_calibration_settings:
        calibrated_parameters.to_file(calibrated_parameters_path)

initialize_FAST_LIVO2(calibrated_parameters) # LiDAR-Visual-Inertial Odometry system
dsclora_model = load_dsclora_model() # Super-Resolution model
scsegamba_model = load_scsegamba_model() # Crack segmentation model
efficientvit_sam_xl0_model = load_efficientvitsam_xl0_model() # Edge refinement model
#unnecessary to load adaptive_pca_sam or 3d_backprojection since it's python code

# Main processing loop
while data_available:
    # Step 1: Acquire synchronized sensor data
    rgb_image, depth_map, imu_data, pose = FAST_LIVO2.get_synchronized_data()

    # Step 2: Enhance image resolution, convert to greyscale for scsegamba
    4x_image = dsclora_model(rgb_image)
    greyscale_4x_image = greyscale(4x_image)

    # Step 3: Perform initial crack segmentation
    initial_crack_mask_logit = scsegamba_model(greyscale_4x_image)

    # Step 4: Assess segmentation confidence
    confidence_score = evaluate_confidence(initial_crack_mask_logit)

    # Step 5: Refine segmentation if confidence is low
    if confidence_score < predefined_threshold:
        initial_prompt = generate_crack_skeleton_prompt(confidence_score)

```

```

    refined_crack_mask = efficientvit_sam_xl0_model (4x_image, initial_prompt)
else:
    refined_crack_mask = confidence_score

# Step 6: Analyze crack morphology
crack_skeleton = extract_skeleton(refined_crack_mask)
crack_widths = estimate_widths(crack_skeleton, high_res_image)

# Step 7: Project crack data into 3D space
crack_3D_data = backproject_to_3D(crack_skeleton, depth_map, pose)

# Step 8: Integrate results into navigation or visualization systems
integrate_with_navigation_system(crack_3D_data, crack_widths)

# Optional: Visualize or store the processed data
visualize_results(high_res_image, refined_crack_mask, crack_3D_data)
store_results(crack_3D_data, crack_widths)

```

4.3 | Adaptive-Normal Tangent Pseudocode

Below is the detailed pseudocode for our implementation of the adaptive normal-tangent crack width estimation algorithm discussed in section 3.6.

```

# Adaptive Normal-Tangent PCA-Based Crack Width Measurement

# Inputs:
# - refined_crack_mask: Binary image with refined crack segmentation
# - segmentation_confidence: Confidence map corresponding to refined_crack_mask
# - curvature_threshold: Threshold to determine high curvature regions
# - min_window_size: Minimum size of the PCA window
# - max_window_size: Maximum size of the PCA window
# - confidence_threshold: Threshold for segmentation confidence to consider measurements

```

Outputs:

- crack_widths: List of measured crack widths at various points along the crack

Step 1: Skeletonize the refined crack mask to obtain the crack centerline

```
skeleton = skeletonize(refined_crack_mask)
```

Step 2: Initialize an empty list to store crack width measurements

```
crack_widths = []
```

Step 3: Iterate over each pixel in the skeleton

for pixel in skeleton:

 # Step 3.1: Extract local neighborhood around the current pixel

```
    local_neighborhood = extract_local_neighborhood(skeleton, pixel, max_window_size)
```

 # Step 3.2: Compute the curvature at the current pixel

```
    curvature = compute_curvature(local_neighborhood)
```

 # Step 3.3: Determine the adaptive window size based on curvature

 if curvature > curvature_threshold:

```
        window_size = min_window_size
```

 else:

```
        window_size = max_window_size
```

 # Step 3.4: Perform PCA on the local neighborhood with the determined window size

```
    tangent_direction, normal_direction = perform_pca(local_neighborhood, window_size)
```

 # Step 3.5: Adjust normal sampling step size based on segmentation confidence

```
    confidence = segmentation_confidence[pixel]
```

 if confidence < confidence_threshold:

```
        continue # Skip measurement if confidence is too low
```

 # Step 3.6: Sample along the normal direction to detect crack edges

```

edge_points = sample_along_normal(pixel, normal_direction, refined_crack_mask)

# Step 3.7: Compute sub-pixel edge localization using interpolation
edge_points_subpixel = interpolate_edges(edge_points, refined_crack_mask)

# Step 3.8: Calculate the crack width as the distance between the interpolated edge points
width = calculate_distance(edge_points_subpixel[0], edge_points_subpixel[1])

# Step 3.9: Store the measured crack width
crack_widths.append((pixel, width))

# Step 4: Return the list of crack widths along with their corresponding pixel locations
return crack_widths

```

4.4 | 3D Backprojection and Crack Localization

Below is pseudocode for our implementation of the 3D backprojection and crack localization algorithm discussed in section 3.7.

```

# Initialize backprojection module parameters
perturbation_samples = 25
intrinsic_matrix = load_intrinsic() # 3x3 matrix K
pose_covariance_model = FAST_LIVO2.get_covariance_model()

# Function to backproject crack pixels to 3D world space
def backproject_to_3D(crack_skeleton, depth_map, pose):
    crack_3D_points = []

    # Decompose SLAM pose into rotation and translation
    R_cw, t_cw = decompose_pose(pose)
    R_wc = R_cw.transpose()
    t_wc = -R_wc @ t_cw

```

```

for pixel in crack_skeleton:
    u, v = pixel

    # Step 1: Normalize pixel coordinate
    pixel_vec = [u, v, 1]
    x_cam = inverse(intrinsics_matrix) @ pixel_vec

    # Step 2: Estimate depth from LiDAR or stereo
    depth = depth_map[u, v]
    if depth <= 0:
        continue # skip invalid depths

    # Step 3: Compute 3D point in camera frame
    X_cam = depth * x_cam

    # Step 4: Transform to world frame
    X_world = R_wc @ X_cam + t_wc

    # Step 5: Initialize uncertainty set
    X_samples = []
    for j in range(perturbation_samples):
        # Apply random perturbation from covariance
        perturbed_pose = sample_pose(pose, pose_covariance_model)
        R_p, t_p = decompose_pose(perturbed_pose)
        R_wp = R_p.transpose()
        t_wp = -R_wp @ t_p
        X_sample = R_wp @ X_cam + t_wp
        X_samples.append(X_sample)

    # Step 6: Estimate uncertainty via covariance
    U = compute_covariance(X_samples)

    # Step 7: Store result

```

```

crack_3D_points.append({
    'position': X_world,
    'uncertainty': U
})

return crack_3D_points

```

4.5 | Results and Prototype Evaluation

For image super-resolution, we initially tested WaveMixSR-V2 against the standard benchmarks lanczos and bicubic resampling strategies on the small Set14 running on an intel 1185g7 laptop cpu. WaveMixSR-V2 performed worse than the standard resampling strategies, suggesting that the model architecture was not high quality and inadequate for the purposes of generalized super resolution. The low WaveMixSR-v2 FPS is due to the file being optimized for the gpu and not the cpu.

Method	PSNR	SSIM	FPS
WaveMixSR-v2	18.97	0.47	1.36
Lanczos	19.25	0.5	206.34
Bicubic	19.3	0.51	328.65

Figure 8: Validation results for WaveMixSR-V2

We then switched to using DSCLoRA due to it being the winner of NTIRE 2025 competition for efficient image super resolution, being the official state of the art real time super resolution model currently. Comparing it to standard torchsr models, torchvision's built in fsrcnn and Real-ESRGAN used in Liu et al's paper, we did a comprehensive evaluation with both non compiled and onnx compiled version of the model to assess performance and quality on V100 gpu node on the Mill on the DIV2K 4x dataset. Based on our results we found that while DSCLoRA had only the third best performance, its was only .4 PSNR lower than the top model and had 10 times faster inference. Compared to Real-ESRGAN, DSCF completely dominates with a PSNR almost 5 greater and a .13 better SSIM, in addition to running 17 times faster.

Method	Avg_PSNR	Avg_SSIM	Avg_Time_ms	FPS
EDSR	29.2478	0.8397	933.05	1.07
RCAN	29.2422	0.8397	548.80	1.82
NinaSR	28.5073	0.8318	137.06	7.30
CARN	28.9759	0.8327	145.19	6.89
Bicubic	26.6952	0.7663	28.95	34.55
Lanczos	26.9350	0.7740	39.61	25.25
DSCF	28.8053	0.8273	91.77	10.90
DSCF_ONNX	28.8053	0.8273	26.17	38.20
RealESRGAN_ONNX	23.8834	0.6895	449.12	2.23

Figure 9: Validation results for DSCF on DIV2K dataset

For SCSegamba, five different attention types (gbc, eca, gbc+eca, sebica, sfa) and three different weighting schemes (original, static weighted fusion, dynamic weighted fusion), and two datasets / three hybrid datasets (TUT, TUT + Crack Conglomerate, Crack Conglomerate) were tested to improve the multi fusion strategy performance and address the lack of learnable attention for these components. For models trained on Crack Conglomerate, the models were trained on a 90/10 train test split, with the train epoch with the best mIoU results calculated from the scikit-image method being used to determine which epoch had the ‘best’ score from the test results. While this led to partially inflated scores (and influenced using a validation split later on), the poor scores from using Crack Conglomerate led to worse performance than the original dataset, negating any potential benefits it may have had over the original dataset. The TUT dataset was trained on the original 70/10/20 train, test and validation split, the same as the original model. Meanwhile, the TUT+ Crack Conglomerate dataset combined the TUT dataset with an 80/10/10 split for crack conglomerate, where 10/90 of the original crack conglomerate training dataset was split off for validation instead of training.

For evaluation of the best metrics, we used the built in evaluation metrics in SCSegamba (mIoU, F1 Score, OIS, ODS, Precision, Recall), in addition to a timer for each evaluation to calculate the exact frames per section for the model to calculate a prediction. To compare to the 2023 method, we faithfully recreated the Niblack + Median filter + Connected Components method described in the paper as a baseline for our model. We used the entire 1096 image crack conglomerate test dataset on a V100 node on the Mill for 50 epochs, batch size=12,

AdamW optimizer and PolyLR scheduling with a Binary Cross-Entropy/Dice Loss of 1:5 validation to ensure extensive, accurate results keeping most of the training the same.

Dataset	Fusion	Attention	mIoU	ODS	OIS	F1	Precision	Recall	FPS
2023_Baseline	N/A	N/A	0.0144	0.0527	0.0527	0.055	0.0283	0.9997	68.24
TUT	original	gbc	0.4818	0.0562	0.0628	0.055	0.0283	0.9954	12.89
TUT	dynamic	gbc	0.484	0.0652	0.0801	0.055	0.0283	0.9951	14.32
TCC	original	gbc	0.4195	0.0616	0.0698	0.0629	0.0325	0.9963	12.95
TCC	dynamic	gbc	0.4219	0.0566	0.0646	0.0625	0.0323	0.9961	14.29
TUT	dynamic	gbc_eca	0.4832	0.0528	0.0676	0.055	0.0283	0.9957	14.26
TCC	dynamic	gbc_eca	0.4191	0.0605	0.0687	0.0629	0.0325	0.9963	14.34
TUT	dynamic	eca	0.4845	0.0543	0.0719	0.055	0.0283	0.9954	15.51
TCC	dynamic	eca	0.4197	0.0635	0.0728	0.063	0.0325	0.9959	15.49
TUT	dynamic	sebica	0.4841	0.0579	0.0714	0.055	0.0283	0.9957	14.75
TCC	dynamic	sebica	0.4194	0.0579	0.0636	0.0629	0.0325	0.9944	14.57
TUT	dynamic	sfa	0.4852	0.0672	0.0849	0.055	0.0283	0.9946	14.66
TUT	dynamic	sfa	0.4202	0.0575	0.0615	0.0628	0.0324	0.9959	14.57
Crack_Conglomerate	original	gbc	0.4183	0.0583	0.0638	0.063	0.0325	0.996	12.74
Crack_Conglomerate	weighted	eca	0.43	0.0539	0.0571	0.0614	0.0317	0.9953	15.09
Crack_Conglomerate	weighted	gbc_eca	0.4291	0.0554	0.0572	0.0616	0.0318	0.9956	14.02
Crack_Conglomerate	weighted	sebica	0.4258	0.0573	0.0608	0.062	0.032	0.9958	14.99
Crack_Conglomerate	weighted	sfa	0.4339	0.0555	0.0635	0.0609	0.0314	0.9957	14.95

Figure 10: Validation results for all variations

As can be seen from the results, the dataset trained on the TUT dataset with the sfa attention has the highest mean IoU score, but eca is only .007 lower and is faster by 1 frame per second in evaluation. ECA is faster by 2.62 frames per second and has .0027 higher mean IoU, such that it is moderately faster and slightly higher performance. The Crack Conglomerate dataset, despite having around eight times as many images and also being the source of evaluation, has worse results than the TUT dataset, suggesting that the images in the Crack Conglomerate dataset may be of worse quality than the TUT dataset. While the model has extremely high precision, the low precision suggests that it is labelling too many pixels as cracks despite that not being the case, potentially suggesting issues with softmax / sigmoid thresholds are something along those lines. This will have to be further investigated later.

While SCSegamba initially appeared to be a suitable choice as a baseline model, further complications have significantly lowered the model's usability. Models exported with onnx are unable to take dynamic shapes as inputs, and large input sizes (e.g. 2048x2048) cause the model export to run out of memory even on a 64gb system, making onnx export infeasible. Despite heavily modifying the source code to make export as a torchscript file, the selective scan function, imported from mamba would require significant reworking and degraded performance compared to the original due to it relying on cuda operations in the source code.

However, even running the non-exported model, while able to accept dynamically sized inputs has significantly degrading performance due to the architecture being unable to accommodate multi-scale features.

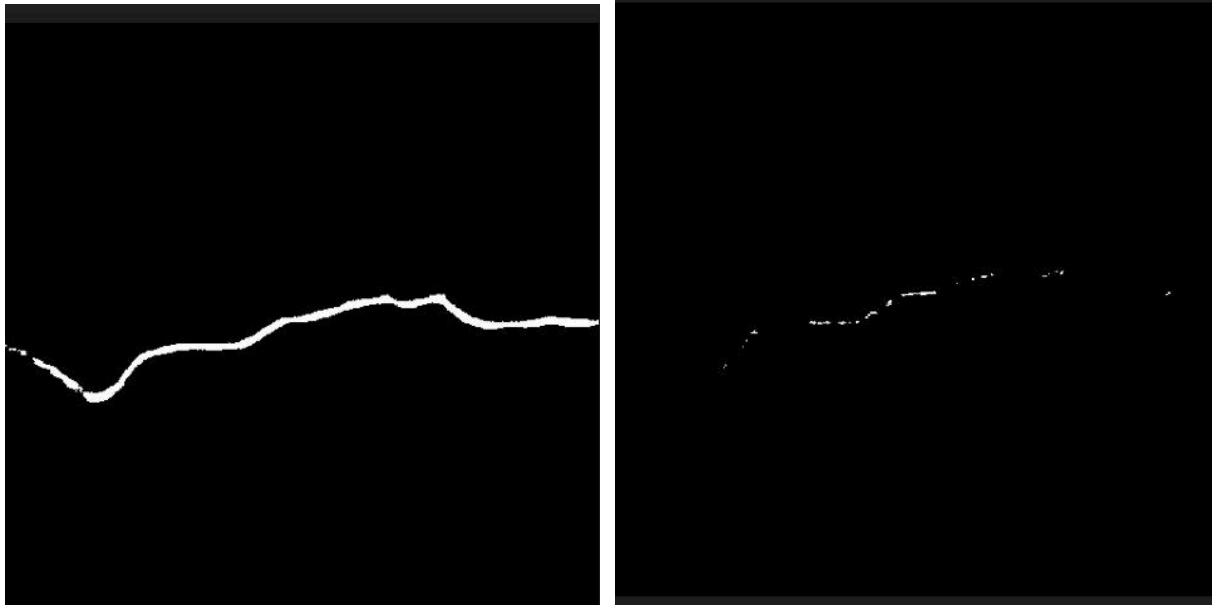


Figure 11: (left): prediction at original scale, (right) prediction at 4x superresolution scale

For usage of EfficientViT-SAM, our initial plan was to use the XL0 model due to it having a much larger 1024x1024 receptive field and patching it across the skeletonized to 6 points of the predictions to predict the mask. The L0 model was completely unable to predict the crack due to being a much lightweight version of the model, while the XL0 version of the model was able to model the crack feature with poor precision.

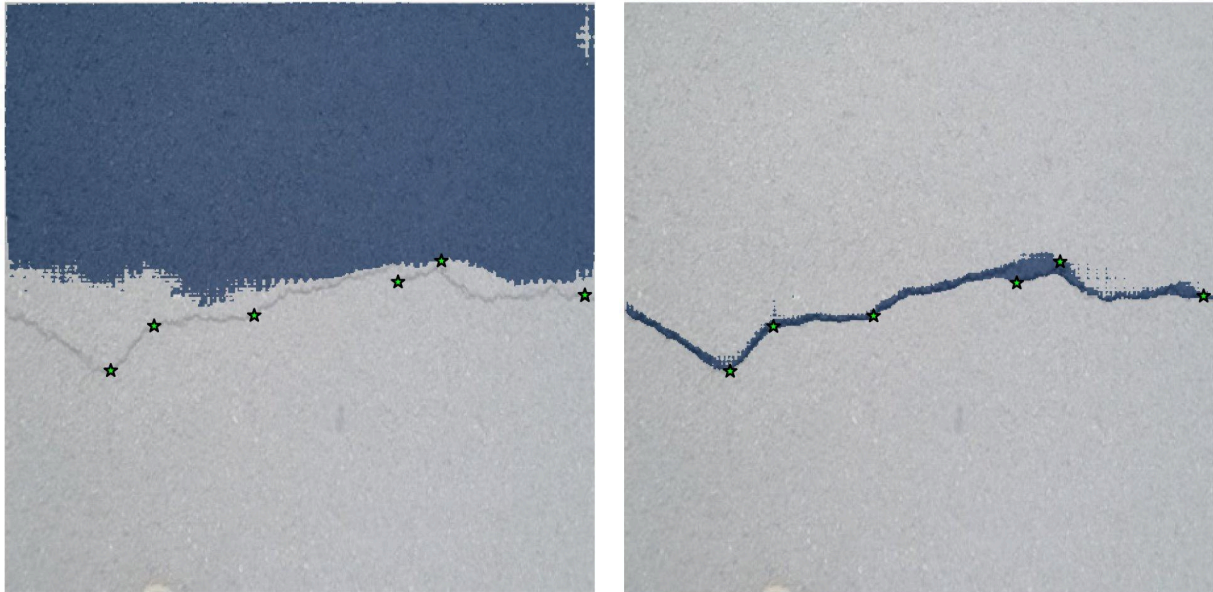


Figure 12: (left): L0 model crack prediction, (right) XL0 model prediction

Since EfficientViT-SAM was a zero shot model with no prior training on crack segmentation, we chose to fine tune the more lightweight decoder on the Crack Conglomerate dataset for 10 epochs, 32 prompts and a batch size of 4 to see if it would improve performance. Due to the complexities of the SAM model, we were unable to get the model exported and instead opted to test the model without exporting it. The results of the fine tuning demonstrated improved performance at predicting the cracks, demonstrating that fine tuning can improve performance.

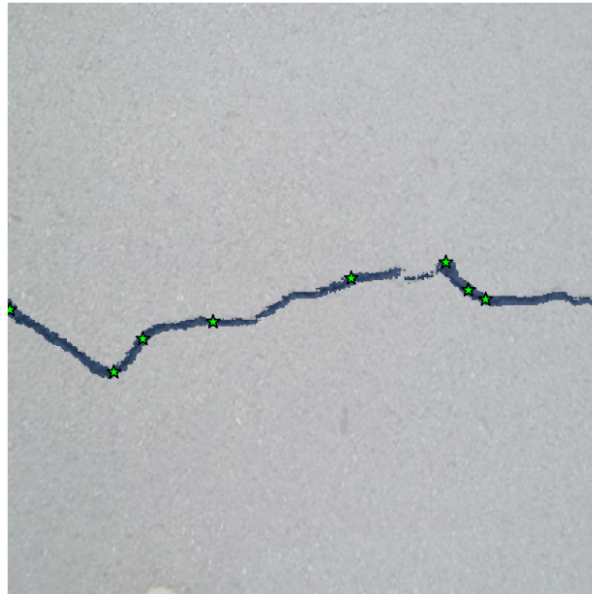


Figure 13: EfficientViT SAM-XL0 fine tuned predictions

Due to not having any pre-existing crack width data to test out our data, we opted to do a direct visual comparison comparing our adaptive PCA method for crack width compared to the original static PCA method using fixed window sizes described in Liu's 2023 paper. Based on our analysis it does appear that adaptive PCA is moderately better at ascertaining the width of the crack pixels with very little increase in overhead.

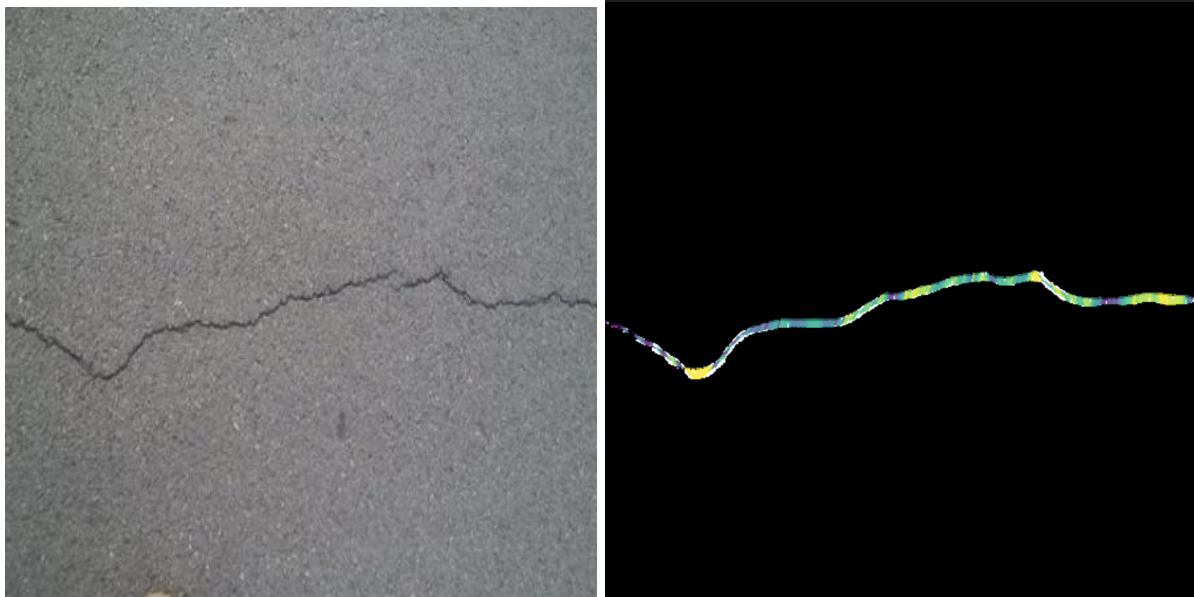


Figure 14: (left) Original crack vs (right) SCSegamba prediction + adaptive PCA output

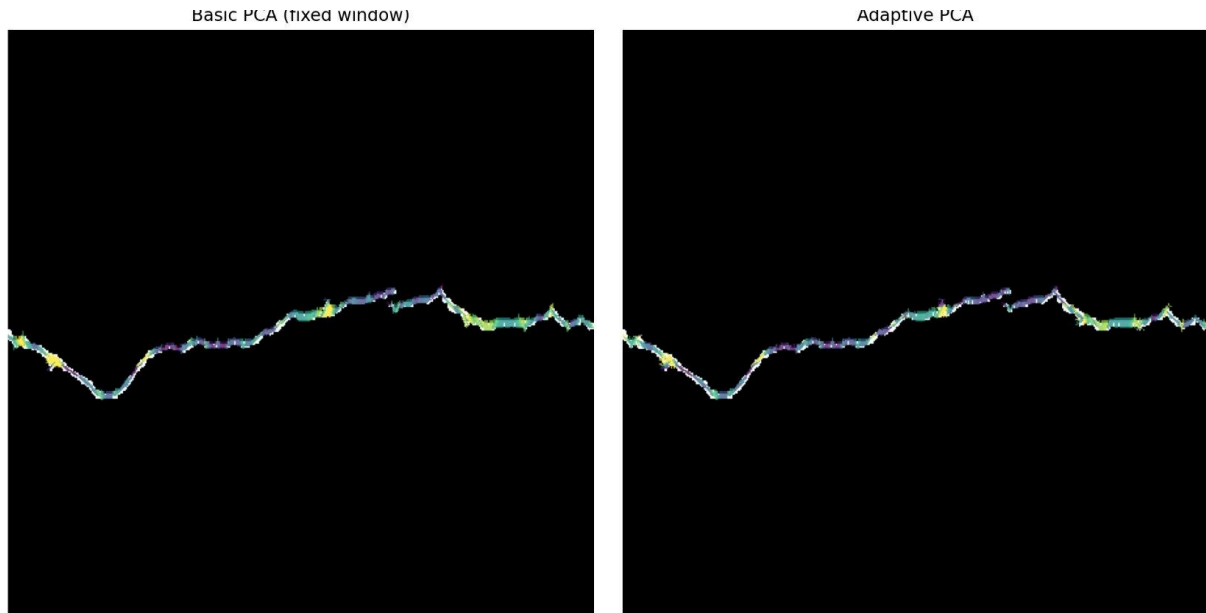


Figure 15: (left) uniform vs (right) adaptive PCA method

For the purposes of this paper, the 3D backprojection was disconnected from the pipeline and tested with synthetic data as a proof of concept. The algorithm was implemented as a Python module which can later be added back to the pipeline. Testing was conducted using three synthetic crack sets of varying complexity. The result was a primitive 3D point cloud projection of the synthetic slabs with embedded cracks. Further work may be necessary to fit this algorithm to the pipeline includes updating function parameters to match output of previous pipeline layers and creating a more thorough algorithm to handle deeper structures. Development was limited by data availability as it was conducted in parallel with other layers of the pipeline and exact data format was not yet available.

3D Projection of Slab with Crack Overlay and Pose Perturbation

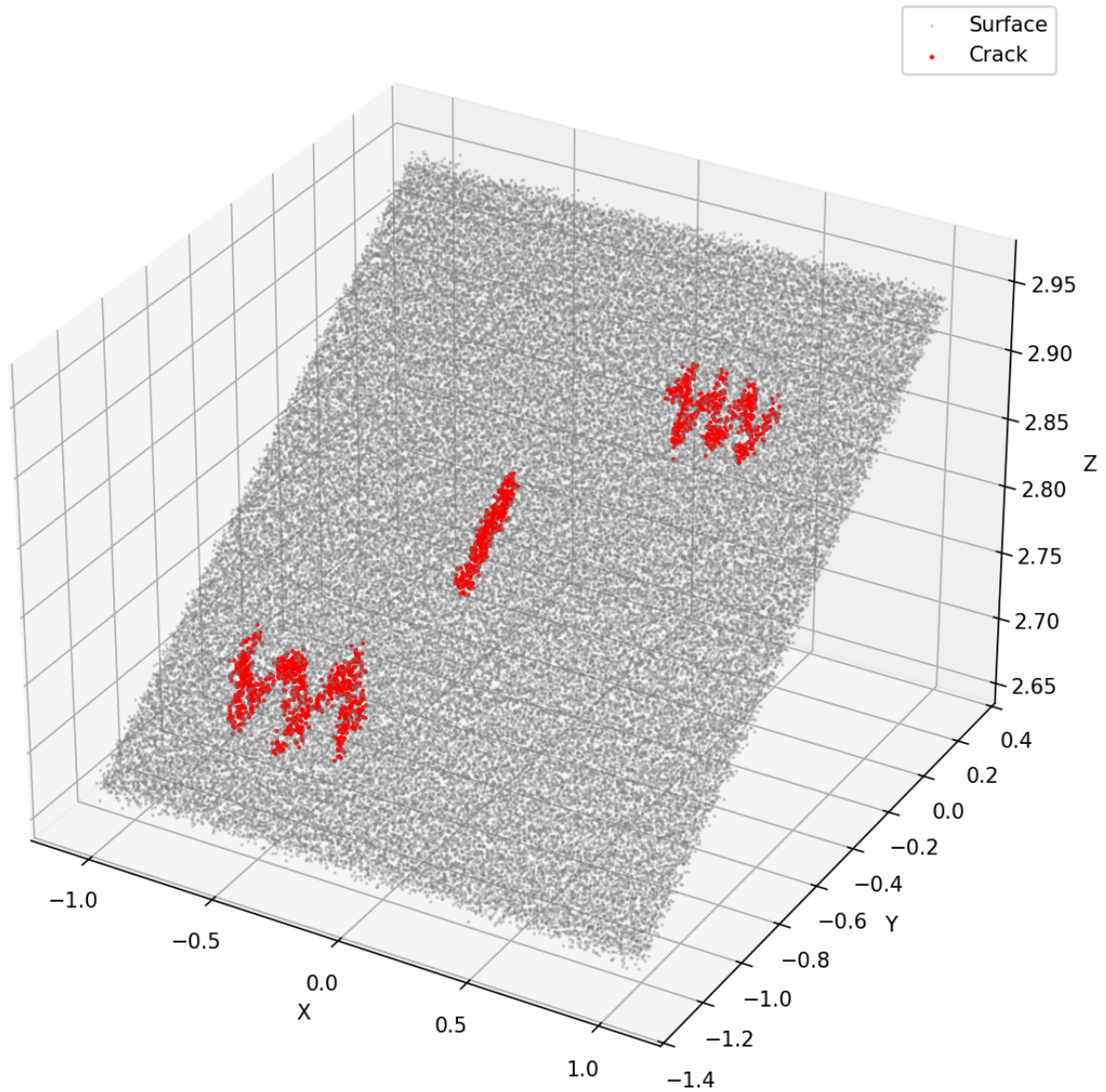


Figure 16: 3D reprojection of synthetic concrete slab (gray points) and with various cracks (red points)

5 | Examples

5.1 | Handheld Real-Time Scan

As a more rudimentary prototype, A technician would use a handheld sensor device with LiDAR, camera, imu and integrated edge computing to detect and measure cracks in real time. As the device moves, it segments cracks and estimates 3D positions and widths on the fly. For a given crack skeleton pixel, the system computes its 3D location and quantifies uncertainty by

sampling small pose perturbations. The adaptive normal–tangent width estimation algorithm then measures sub-pixel crack widths in metric units.

This version of the application will be particularly useful in situations where a larger, expensive drone-based setup is not practical. Example use cases include indoor structures, constrained spaces like coal shafts, or when an inspector is already on-site and can use the tool to alert them to problems and quickly verify with hand tools.

5.2 | Drone-Based Bridge Inspection

This would be a fully fleshed out version of the prototype, and may have a degree of autonomous navigation built in. A drone equipped with a calibrated RGB-D camera, LiDAR, and IMU performs a scan beneath a highway overpass. FAST-LIVO2 generates real-time pose estimates, while the crack segmentation and refinement pipeline would generate the crack segments.

A crack pixel $p = (u, v)$ is backprojected into 3D using the intrinsic matrix K and depth value d , resulting in:

$$\hat{x}_{cam} = \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

This point is transformed into world coordinates using the camera pose, allowing spatially accurate crack mapping, and would be repeated to build a 3D crack map on the surface. Converting this into a Nav2 package, Nav2 built in path-planning would automatically navigate the robot with human supervision to map the entire structure and spit out a 3D scan in real time.

This architecture would be designed for vast, non-human accessible environments like multi-story bridges, smokestacks on power plants, piers, oil rig underwater support structures, and large dams, utilizing the mobility and autonomous features that are necessary for these much harder infrastructure monitoring tasks where the drone is the only accessible verification source.

6 | Conclusion

In this work, we presented a unified, real-time crack detection and 3D localization system for structural health monitoring using SLAM and deep learning, optimized for edge deployment on gpu accelerated Jetson devices. By integrating FAST-LIVO2 for robust multi-sensor SLAM, DSCLoRA for efficient image super-resolution, SCSegamba for lightweight and accurate segmentation, and EfficientViT-SAM-XL0 for refinement, our pipeline balances high

performance with practical deployability. We further introduced an adaptive normal–tangent PCA-based algorithm for precise crack width estimation and developed a 3D backprojection module with uncertainty quantification for reliable crack localization in spatial context.

Compared to prior work, our system will significantly improve efficiency without compromising accuracy, making it suitable for real-world field deployment via handheld devices or autonomous drones. Architectural improvements like lightweight attention and dynamic weighting helped marginally improve efficiency and moderately boosted efficiency for SCSegamba, and the crack conglomerate dataset, despite having many more images than TUT had significantly worse performance. However, due to architectural flaws in SCSegamba only being designed for images at the scale trained, as well as the Selective Scan architecture making it impossible to export to a portable pt model without heavily rewriting the function to lose gains in CUDA efficiency, we will have to benchmark other models like EfficientCrackNet, CrackEdgeSAM and HrSegNet to compare performance and efficiency to determine if the model should be replaced outright, or if heavy modifications for scale invariance should be done. EfficientViT-SAM demonstrated improved crack segmentation with fine tuning, demonstrating the importance of light fine tuning to improve task-specific performance for crack segmentation. We implemented a primitive pose-aware 3D backprojection module to reconstruct cracks within a point cloud. Though disconnected from the full pipeline due to integration timing and dataset availability, this module was evaluated individually and proved robust in various test cases. Future works should be able to easily integrate this module with minor adaptations.

The planned modular ROS2-based architecture will allow for enhanced accessibility and integration with future navigation or inspection frameworks. Moving forward, our goal is to address the issue with multi-scale scale-representation with the segmentation foundation model system and finding the proper dataset, followed by full 3d point cloud integration.

7 | References

- [1] P. Deng *et al.*, “Unified Few-shot Crack Segmentation and its Precise 3D Automatic Measurement in Concrete Structures,” *arXiv.org*, 2025. <https://arxiv.org/abs/2501.09203>.
- [2] C.-Q. Feng, B.-L. Li, Y.-F. Liu, F. Zhang, Y. Yue, and J.-S. Fan, “Crack assessment using multi-sensor fusion simultaneous localization and mapping (SLAM) and image super-resolution for bridge inspection,” *Automation in Construction*, vol. 155, p. 105047, Aug. 2023, doi: <https://doi.org/10.1016/j.autcon.2023.105047>.
- [3] T. Du, Y. H. Zeng, J. Yang, C. Z. Tian, and P. F. Bai, “Multi-sensor fusion SLAM approach for the mobile robot with a bio-inspired polarised skylight sensor,” *IET Radar, Sonar & Navigation*, Sep. 2020, doi: <https://doi.org/10.1049/iet-rsn.2020.0260>.
- [4] J. L. Carrivick, M. W. Smith, and D. J. Quincey, *Structure from Motion in the Geosciences*. Chichester, UK: John Wiley & Sons, Ltd, 2016. doi: <https://doi.org/10.1002/9781118895818>.
- [5] P. Jeevan, N. Nixon, and A. Sethi, “WaveMixSR-V2: Enhancing Super-resolution with Higher Efficiency,” *arXiv.org*, 2024. <https://arxiv.org/abs/2409.10582>.
- [6] H. Liu, C. Jia, F. Shi, X. Cheng, and S. Chen, “SCSegamba: Lightweight Structure-Aware Vision Mamba for Crack Segmentation in Structures,” *arXiv.org*, 2025. <https://www.arxiv.org/abs/2503.01113>.
- [7] C. Zheng *et al.*, “FAST-LIVO2: Fast, Direct LiDAR-Inertial-Visual Odometry,” *arXiv.org*, 2024. <https://arxiv.org/abs/2408.14035>.
- [8] X. Sun, J. Liu, H. T. Shen, X. Zhu, and P. Hu, “On Efficient Variants of Segment Anything Model: A Survey,” *arXiv.org*, 2024. <https://arxiv.org/abs/2410.04960>.
- [9] Z. Zhang, H. Cai, and S. Han, “EfficientViT-SAM: Accelerated Segment Anything Model Without Accuracy Loss,” *arXiv.org*, 2024. <https://arxiv.org/abs/2402.05008>.
- [10] H. Liu, J. Yang, X. Miao, C. Mertz, and H. Kong, “CrackFormer Network for Pavement Crack Segmentation,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 9, pp. 9240–9252, Sep. 2023, doi: <https://doi.org/10.1109/tits.2023.3266776>.
- [11] X. Jiang, X. Wan, K. Zhu, X. Qiu, and Z. Fang, “Distribution-aware Noisy-label Crack Segmentation,” *arXiv.org*, 2024. <https://arxiv.org/abs/2410.09409>.
- [12] Y. Wu, S. Li, J. Li, Y. Yu, J. Li, and Y. Li, “Deep learning in crack detection: A comprehensive scientometric review,” *Journal of Infrastructure Intelligence and Resilience*, vol. 4, no. 3, p. 100144, Feb. 2025, doi: <https://doi.org/10.1016/j.iintel.2025.100144>.
- [13] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.
- [14] Qijun, Luo & Ge, Baozhen & Qingguo, Tian. (2019). A fast adaptive crack detection algorithm based on a double-edge extraction operator of FSM. *Construction and Building Materials*. 204. 244-254. 10.1016/j.conbuildmat.2019.01.150.
- [15] Chai, X., Zhang, Y., Zhang, Y., Cheng, Z., Qin, Y., Yang, Y., & Song, L. (2025). Distillation-Supervised Convolutional Low-Rank Adaptation for Efficient Image Super-Resolution. *arXiv preprint arXiv:2504.11271*. <https://doi.org/10.48550/arXiv.2504.11271>

8 | Appendix

Appendix A) Code Repository

Source code and data are available at: <https://github.com/ajc3xc/masters-thesis-project/>